

QRDecomposition – Mathematica vs Mathilda

A side-by-side audit of `QRDecomposition[m]` covering exact / symbolic inputs, machine-precision inputs, and arbitrary-precision (MPFR) inputs. The outputs in the Mathilda columns were produced by the build of Mathilda at `f695028` on 2026-05-22; the Mathematica behaviour was sampled via `wolframscript` against Wolfram Mathematica 13 and cross-checked with the documented contract in Wolfram Documentation Center for `QRDecomposition`.

The two systems agree on the public contract, the “thin” QR shape, the permutation-matrix convention, and the satisfied identity. They differ in (i) how aggressively the exact pipeline canonicalises symbolic output, (ii) how rank-deficient arbitrary-precision input is handled, and (iii) whether `TargetStructure -> "Structured"` is implemented.

1. API surface

Feature	Mathematica	Mathilda
<code>QRDecomposition[m] -> {q, r}</code>	yes	yes
Identity: <code>m == ConjugateTranspose[q] . r</code>	yes	yes
Thin / rank-revealing shape (<code>Length[q] == Length[r] == MatrixRank[m]</code>)	yes	yes
Pivoting -> <code>True -> {q, r, p}, m . p == ConjugateTranspose[q] . r</code>	yes	yes
<code>TargetStructure -> "Structured"</code> (returns <code>OrthogonalMatrix / UpperTriangularMatrix / PermutationMatrix</code>)	yes	no – option recognised, call left unevaluated until those wrapper heads exist
<code>TargetStructure -> "Dense"</code> (default)	yes	yes
<code>Method -> ...</code>	yes	no – option not parsed; Mathilda’s kernel choice is fully determined by input precision via <code>qr_dispatch</code>
Listable / threading over arrays	n/a	n/a

Both systems issue `QRDecomposition::matrix` for non-rectangular arguments and return unevaluated.

Conventions agree: rows of `q` are orthonormal (unitary in the Hermitian inner product for complex `m`), `r` is upper trapezoidal.

2. Symbolic / exact inputs

2.1 Exact integer `{{1, 2}, {3, 4}}`

Mathematica

```
QRDecomposition[{{1, 2}, {3, 4}}]
=> {{{1/Sqrt[10], 3/Sqrt[10]}, {3/Sqrt[10], -(1/Sqrt[10])}},
     {{Sqrt[10], 7 Sqrt[2/5]}, {0, Sqrt[2/5]}}}
```

Mathilda

```
QRDecomposition[{{1, 2}, {3, 4}}]
=> {{{1/Sqrt[10], 3/Sqrt[10]},
     {3/5/Sqrt[2/5], -1/5/Sqrt[2/5]}},
     {{Sqrt[10], 14/Sqrt[10]},
     {0, Sqrt[2/5]}}}
```

Algebraically identical ($3/5/\text{Sqrt}[2/5] == 3/\text{Sqrt}[10]$, $14/\text{Sqrt}[10] == 7 \text{Sqrt}[2/5]$), but Mathematica applies its full simplifier on the result while Mathilda stops at `Together / Expand` on each entry. The reconstruction identity $\text{Transpose}[q] \cdot r == m$ holds in both systems.

2.2 Rank-deficient {{1,2,3},{4,5,6},{7,8,9}}

Both systems emit the same shape – q is 2×3 , r is 2×3 , matching $\text{MatrixRank}[m] == 2$.

Mathilda output:

```
q = {{1/Sqrt[66], 4/Sqrt[66], 7/Sqrt[66]},
     {3/Sqrt[11], 1/Sqrt[11], -1/Sqrt[11]}}
r = {{Sqrt[66], 78/Sqrt[66], 90/Sqrt[66]},
     {0, 3/Sqrt[11], 6/Sqrt[11]}}
Transpose[q] . r == {{1,2,3},{4,5,6},{7,8,9}} (* exact *)
```

Mathematica returns the same factorisation up to its tighter canonical form on the radicals.

2.3 Free-symbolic {{a, b}, {c, d}}

Mathematica produces a clean closed form using $(a d - b c) / \text{Sqrt}[a^2 + c^2]$ for the $r[2,2]$ entry and a simplified q .

Mathilda also satisfies the identity – $\text{Simplify}[\text{Transpose}[q] \cdot r - m] == \{\{0,0\},\{0,0\}\}$ – but the printed entries are the raw MGS expressions, including triple-nested radicals such as

```
Sqrt[(b - (a ((a b)/Sqrt[a^2+c^2] + (c d)/Sqrt[a^2+c^2]))/Sqrt[a^2+c^2])^2
+ (d - (c ((a b)/Sqrt[a^2+c^2] + (c d)/Sqrt[a^2+c^2]))/Sqrt[a^2+c^2])^2]
```

This is `Together`-canonical but not factored. Wrapping the Mathilda call in `Simplify[QRDecomposition[...]]` is the recommended workaround.

Conclusion (symbolic). Both systems return correct exact factorisations. Mathematica's q and r print materially shorter on free-symbolic input because Mathematica runs `Simplify` over the result; Mathilda deliberately stops at `Together/Expand` for cost reasons (`Simplify` on intermediate Gram-Schmidt forms is super-linear in the matrix size). The numerical content is the same.

2.4 Complex exact {{1+I, 2}, {3, 4-I}}

Both systems use the Hermitian inner product $\langle a, b \rangle = \text{Sum}_i \text{Conjugate}[a_i] b_i$. Mathematica's exact pipeline simplifies $\text{Conjugate}[\text{Sqrt}[11]] \rightarrow \text{Sqrt}[11]$. Mathilda's `Conjugate` does not push through symbolic $\text{Sqrt}[\text{positive integer}]$ during exact evaluation, so its q carries $\text{Conjugate}[3/\text{Sqrt}[11]]$ residues. The identity still holds in the machine-precision complex case ($\{\{1.0+I, 2.0\}, \{3.0, 4.0-I\}\}$) because the rationalise-numeralise pipeline collapses the residues at the end.

3. Machine-precision inputs

For any matrix carrying at least one `Real` leaf whose precision is ≤ 53 bits, Mathilda routes to `qr_machine_dispatch` in `src/linalg/qrdecomp_machine.c`, which wraps LAPACK (`dgeqrf` / `dgeqp3` for real, `zgeqrf` / `zgeqp3` for complex, plus `dorgqr` / `zungqr` to form `q`). LAPACK is discovered through the four-tier autodetect ladder documented in `src/linalg/lapack.h` (Apple Accelerate, `pkg-config lapacke`, system `lapacke`, graceful fallback to the symbolic kernel).

Mathematica also dispatches to LAPACK for `MachinePrecision` input, so the algorithms are identical except for backing library.

3.1 $\{\{1.2, 2.3, 3.4\}, \{2.3, 4.5, 5.6\}, \{3.2, 7.6, 6.5\}\}$

```
{q, r} = QRDecomposition[A];
Chop[Transpose[q] . r - A]
=> {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}
Chop[q . Transpose[q]]
=> {{1.0, 0, 0}, {0, 1.0, 0}, {0, 0, 1.0}}
```

Both Mathematica and Mathilda satisfy reconstruction to machine epsilon; the orthogonality residual is identical to within rounding because both backings call the same LAPACK routines on the same data.

3.2 Non-square machine matrices

Input shape	q shape	r shape	Both correct?
$n \times p$ with $n > p$ (tall)	$p \times n$	$p \times p$	yes
$n \times p$ with $n < p$ (wide)	$n \times n$	$n \times p$	yes
Rank-deficient $n \times p$	$k \times n$	$k \times p$, $k = \text{rank}$	yes

LAPACK's rank cutoff $\max(m, n) * \text{eps} * |R[0, 0]|$ is used by both systems. For a rank-deficient input *without Pivoting* $\rightarrow$ `True` LAPACK still returns full `Q/R`; Mathilda truncates to numerical rank after the call, matching Mathematica.

3.3 Complex machine $\{\{1.0+I, 2.0\}, \{3.0, 4.0-I\}\}$

```
q = {{0.301511 - 0.301511*I, 0.904534},
      {0.286039 + 0.858116*I, 0.190693 - 0.381385*I}}
r = {{3.31662, 4.22116 - 1.50756*I},
      {0.0, 0.953463}}
ConjugateTranspose[q] . r - m == 0 (to machine epsilon)
```

Output matches Mathematica modulo sign conventions on the reflectors (LAPACK chooses a deterministic sign that depends on the input scaling; Mathematica's published output uses the same convention).

4. Arbitrary-precision (MPFR) inputs

For any inexact input whose minimum leaf precision is > 53 bits, Mathilda routes to `qr_mpfr_dispatch` in `src/linalg/qrdecomp_mpfr.c`. This is a hand-rolled **Householder QR over column-major MPFR arrays** (paired re/im planes for complex; no MPC dependency, same convention as the eigen kernels). Column pivoting follows Businger-Golub; numerical rank uses the cutoff $|R[i, i]| < 2^{-(\text{bits}/2)} * |R[0, 0]|$.

Mathematica's arbitrary-precision QR uses high-precision Givens rotations through its `Internal\HighPrecisionNumerics` stack. Both guarantee reconstruction-error scaling of $\text{order}2^{-(\text{bits})}$ for full-rank input.

4.1 Symmetric full-rank $\{\{4,1,2\},\{1,3,0\},\{2,0,5\}\}$ at 80 bits

```
mp = {{N[4,80], N[1,80], N[2,80]},  
      {N[1,80], N[3,80], N[0,80]},  
      {N[2,80], N[0,80], N[5,80]}};  
{q, r} = QRDecomposition[mp];
```

Mathilda's `q[1,1]` and `r[1,1]` carry 80-bit precision (~ 24 decimal digits printed). The reconstruction residual entries are of order $2^{(-80)} \sim 8.3 * 10^{(-25)}$. The Mathematica output, when reduced to 80 bits, agrees to within last-bit rounding on every entry.

4.2 Mixed precision (some MachinePrecision, some MPFR)

Both systems take the minimum input precision as the working precision. `qr_dispatch` calls `common_scan_inexact(m)`; if `min_bits <= 53` it goes machine, otherwise MPFR.

4.3 Rank-deficient MPFR input

The MPFR kernel detects rank deficiency two ways:

1. **With Pivoting -> True:** the Businger-Golub pivot makes the R diagonal monotone non-increasing, so a small diagonal terminates the rank count cleanly. Output dimensions exactly track `MatrixRank` – same as Mathematica.
2. **With Pivoting -> False:** the kernel runs Householder reflectors as usual; when the residual column norm at step `k` rounds to a finite-but-tiny value, the reflector still completes and the rank-truncation happens implicitly through the numerical-rank cutoff applied to $|R[i, i]|$. The kernel returns a rank-`k` `{q, r}` with no warning. This is the **common case** for exactly rank-deficient input that exposes through rounding; for the rare path where a residual norm rounds to exactly zero, the kernel returns NULL and `qr_dispatch` falls through to the symbolic Modified Gram-Schmidt pipeline with the one-shot warning

QRDecomposition: MPFR fast path hit rank-deficient input without pivoting; falling back to symbolic kernel.

Mathematica reports no such warning – it stays in high-precision Givens throughout.

Recommended Mathilda usage on rank-deficient arbitrary precision matrices: pass `Pivoting -> True`. This avoids the fallback path entirely and produces a numerically robust, monotone-R-diagonal result.

5. Pivoting

Both systems implement column pivoting via the same Businger-Golub selection (pick the column with the largest residual squared norm at each step) and produce the same permutation matrix convention: $P[\text{perm}[j], j] = 1$, satisfying $m \cdot p == \text{ConjugateTranspose}[q] \cdot r$.

5.1 Exact pivoting $\{\{1, 2\}, \{3, 4\}\}$, Pivoting -> True

```
{{{1/Sqrt[5], 2/Sqrt[5]}, {-2/Sqrt[5], 1/Sqrt[5]}},  
 {{2 Sqrt[5], 7/Sqrt[5]}, {0, 1/Sqrt[5]}},  
 {{0, 1}, {1, 0}}}
```

Identical to Mathematica modulo Mathematica's tighter radical simplification. Column 2 (norm `Sqrt[20]`) was selected first.

5.2 Machine 5x4 pivoting

R-diagonals come out in strictly decreasing magnitude: {4.06202, 3.76738, 3.1174, 2.0831}, matching Mathematica.

6. Edge cases

Input	Mathematica	Mathilda
{{0, 0}, {0, 0}}	{{}, {}} (empty q, r)	{{}, {}} – match
{{5}}	{{{1}}, {{5}}}	{{{1}}, {{5}}} – match
Non-rectangular	QRDecomposition::matrix, unevaluated	QRDecomposition::matrix, unevaluated
Pivoting -> Foo	option error, unevaluated	QRDecomposition::opts, unevaluated

7. Summary of behavioural deltas

The following table is the punch list of where the two systems *intentionally* diverge:

Area	Difference
Free-symbolic output canonical form	Mathilda stops at Together / Expand on each entry; Mathematica runs full Simplify . Wrap in Simplify[...] on the Mathilda side if a clean closed form is required.
Complex exact Conjugate[Sqrt[n]]	Mathilda leaves a Conjugate[Sqrt[11]] residue inside q; Mathematica simplifies to Sqrt[11] . Both reconstruct m after numericalisation.
TargetStructure -> "Structured"	Mathematica returns OrthogonalMatrix / UpperTriangularMatrix / PermutationMatrix wrappers. Mathilda leaves the call unevaluated because those head wrappers are not yet implemented. TargetStructure -> "Dense" (the default) is identical.

Area	Difference
Rank-deficient MPFR without pivoting	Mathematica stays in Givens-precision. Mathilda's MPFR kernel bails out (warns once) and the symbolic kernel finishes the job correctly. Pass <code>Pivoting -> True</code> to keep the MPFR fast path engaged.
Method -> ... option	Not supported in Mathilda; Mathilda's kernel choice is fully determined by input precision via <code>qr_dispatch</code> .

Areas where the two systems agree:

- Public API (`QRDecomposition[m]`, `Pivoting -> True`).
- Thin / rank-revealing output shape.
- Reconstruction identity (`m == ConjugateTranspose[q] . r, m . p == ConjugateTranspose[q] . r` under pivoting).
- LAPACK backing on machine precision (`dgeqrf` / `dgeqp3` / `dorgqr` for real; `zgeqrf` / `zgeqp3` / `zungqr` for complex).
- Householder-with-pivoting algorithm on arbitrary precision.
- Permutation-matrix convention `P[perm[j], j] = 1`.
- Edge cases on zero, 1x1, and non-rectangular input.

8. Where to look in the source

- `src/linalg/qrdecomp.h` – public surface and contract.
- `src/linalg/qrdecomp.c` – option parsing, `qr_dispatch` router, symbolic Modified Gram-Schmidt kernel (`qr_symbolic_core`, `qr_symbolic_dispatch`).
- `src/linalg/qrdecomp_machine.c` – LAPACK machine-precision path (`qr_machine_dispatch`).
- `src/linalg/qrdecomp_mpfr.c` – MPFR Householder fast path (`qr_mpfr_dispatch`).
- `src/linalg/lapack.h` / `lapack.c` – LAPACK ABI wrappers and the four-tier autodetect ladder shared with the LU / Eigen / Pseudo Inverse kernels.
- `src/common.h` / `common.c` – the `common_scan_inexact` / `common_rationalize_input` / `common_numericalize_result` shared inexact-pipeline used by QR, LU, Eigen, and PseudoInverse.
- `tests/test_qrdecomposition.c` – exact / symbolic tests.
- `tests/test_qrdecomposition_machine.c` – LAPACK path tests.
- `tests/test_qrdecomposition_mpfr.c` – MPFR path tests.
- `tests/bench_qr.c` – timing harness.
- `docs/spec/builtins/linear-algebra.md` (`## QRDecomposition`) – user-facing spec.
- `LU_DECOMPOSITION_COMPARISON.md` – the companion comparison document for `LUDecomposition`.